

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Portføljeopgave 1

E25: Algoritmer og Datastrukturer

Alt kode kan kopieres fra dette dokument, men kan også findes i de vedhæftede filer (P1Exercises.cpp, P1Exercises.h og main.cpp).

Dette er kørt fra main, med de tests som er blevet specificeret i opgavebeskrivelsen. Derfor: hvis man køre main vil man få denne udskrift.

```
lina@li:~/Workspace/ADA/P1$ ./main
P1 works
----- OPGAVE 1 -----
N: 8
odd squares sum of N = 84
----- OPGAVE 2 -----
see document for solution
----- OPGAVE 3 -----
String of numbers: 82842605
Method returned: 1
----- OPGAVE 4 -----
original array:
23, 56, 22, 11, 65, 89, 3, 44, 87, 910, 45, 35, 98, 0,
returned array:
89, 3, 35, 127,
----- OPGAVE 5 -----
see document for solution
----- OPGAVE 6 -----
N: 12
sum3(N) = 30
N: 14
sum3(N) = 30
----- OPGAVE 7 -----
Z: 6561
Z = 9^4
```

```
----- OPGAVE 8 -----
see document for solution
----- OPGAVE 9 -----
see document for solution
----- OPGAVE 10 -----
N: 32
Log2(N) = 5
N: 4096
Log2(N) = 12
----- OPGAVE 11 -----
original votes:
7, 4, 3, 5, 3, 1, 6, 4, 5, 1, 7, 5, 98,
bestCandidate: 5
maxVotes: 3
returned = -1
-----
original votes:
7, 7, 7, 7, 7, 6, 4, 5, 1, 7, 5, 98,
bestCandidate: 7
maxVotes: 7
returned = 7
----- OPGAVE 11: Boyer Moore -----
original votes:
7, 4, 3, 5, 3, 1, 6, 4, 5, 1, 7, 5, 98,
returned = -1
-----
original votes:
7, 7, 7, 7, 7, 7, 6, 4, 5, 1, 7, 5, 98,
returned = 7
Which exercise do you want to run?
```

Opgave 1

```
int P1Exercises::oddSquares(int N) {
    int sqrSum = 0;
    if(N <= 0) {
        return 0; //base case, or non-positive sum
    }
    if((N & 1) == 0) { //if LSB is 0 then n is even
        N--;
    }
    return N*N + oddSquares(N-2); //only recursive on odd squares
}
```

Udskrift fra main

```
----- OPGAVE 1 -----
N: 8
odd squares sum of N = 84
```

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 2

Nedenfor er udprint for metoden kaldt med $N = 10$

```
----- OPGAVE 2 -----
k: 32
k: 32
k: 32
k: 32
j: 15
k: 32
k: 32
k: 32
k: 32
j: 15
k: 32
k: 32
k: 32
j: 15
k: 32
k: 32
x: 512
i: 15
y: 100
x + y = 612

int myMethod(int N){
    int x = 0; int y = 0;
    int j;
    int k;
    int i;
    for(i = 0; i < N; i++){
        for(j = 0; j < N; j++){
            for (k = 0; k < N*sqrt(N); k++){
                x++;
            }
            //std::cout<<"k: " << k <<std::endl;
            j*=2;
        }
        //std::cout<<"j: " << j<< std::endl;
        i += i;
    }
    std::cout<<"x: " << x<< std::endl;
    //std::cout<<"i: " << i<<std::endl;
    for(int i = 0; i < N*N; i++){
        y++;
    }
    std::cout<< "y: " << y << std::endl;
    return x+y;
}

----- OPGAVE 2 -----
x: 2250
y: 400
x + y = 2650

----- OPGAVE 2 -----
x: 9108
y: 1600
x + y = 10708

----- OPGAVE 2 -----
x: 35084
y: 6400
x + y = 41484

Køres algoritmen på:
N = 20, N = 40 og N = 80
ses det at værdien af x
stiger eksponentielt
```

K-løkke: $O(N \cdot \sqrt{N}) \Rightarrow O(N^{3/2})$

N er 10 og $10 \cdot \sqrt{10} \Rightarrow 31.6 \Rightarrow 32$ (N er en int, der rundes op)

Der bliver lagt $k = 32$ til x for hver fuld iteration af K-løkken.

k-løkke har 512 inkrementer ($x = 512$)

J-løkke: $O(\log N)$

J fordobles hver gang, hvilket medfører at antallet af iterationer "eksponentielt formindskes" iterationer af j kan skrives som:

$j = 0 \rightarrow j = 1 \rightarrow j = 2 \rightarrow j = 4 \rightarrow j = 8 \rightarrow j = 16$ ($j > N = 10$, så løkken fejler her)

4 iterationer

I-løkke: $O(\log N)$

$i = 0 \rightarrow i = 1 \rightarrow i = 2 \rightarrow i = 4 \rightarrow i = 8 \rightarrow i = 16$ ($i > N = 10$, så løkken fejler her)

4 iterationer

Samlet produktet: $O(N^{3/2}) * O(\log N) * O(\log N) \Rightarrow O(N^{3/2} * (\log N)^2)$

dette er en logaritmisk formindskning

Den sidste for-løkke har tidskompleksitet **$O(N^2)$ altså kvadratisk.**

Denne tidskompleksitet er højere end de forgående; hvorfor det er den der er gældende for algoritmen.

myMethod $\rightarrow O(N^2)$ tidskompleksitet

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 3

```
bool additive(std::string s){  
    if(s.size() < 3){ //as long as there is a string of at least 3  
        return false;  
    }  
  
    int first = s[0]- '0'; //48 is ascii value of zero  
    int second = s[1]- '0';  
    int third = s[2]- '0';  
  
    if(third == second + first){ //condition met  
        return true;  
    }  
    return additive(s.substr(1)); //progress  
}
```

```
----- OPGAVE 3 -----  
String of numbers: 82842605  
Method returned: 1
```

"1" = true

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 4

```
std::vector<int> findClosestPot(std::vector<int> arr){
int arrSize = arr.size();
    if(arrSize == 0){ //as long as there is a string
        return {-1}; //error?
    }

    /*største potens*/
    int maxElement = *std::max_element(arr.begin(), arr.end()); //value of
    int potMax = maxElement*3; //hvis der er 3 af den samme maks værdi

    /*pot-2 array*/
    std::vector<int> potTwo;
    int inc = 0;
    int power = 0;
    while( power <= potMax){
        power = pow(2, inc);
        potTwo.push_back(power);
        //std::cout<<"power: "<<power<<std::endl;
        inc++;
    }

    /* init return array*/
    int bestPower = potTwo[0];
    int closestSum = arr[1]+arr[2]+arr[3];
    int bestDiff = closestSum - bestPower;
    std::vector<int> clstSumArr = {arr[1], arr[2], arr[3], closestSum};

    /*find closest sum*/
    for(int i = 0; i <= arrSize-1; i++){
        for(int j = 0; j <= arrSize-1; j++){
            for(int k = 0; k <= arrSize-1; k++){

                if(arr[i] != arr[j] && arr[i] != arr[k] && arr[j] != arr[k]){ //dont use same
number twice

                    int sum = arr[i] + arr[j] + arr[k];

                    for(int power : potTwo){ //check against each power

                        int diff = abs(sum - power); //get positive difference

                        if(diff < bestDiff){
                            bestDiff = diff; //update best difference
                            clstSumArr = {arr[i], arr[j], arr[k], power};
                        }
                    }
                }
            }
        }
    }

    return clstSumArr;
}
```

----- OPGAVE 4 -----

original array:

23, 56, 22, 11, 65, 89, 3, 44, 87, 910, 45, 35, 98, 0,

returned array:

89, 3, 35, 128,

5. semester civilingeniør i robotteknologi bachelor

Det kan ses at hvis man fordobler værdien af N stiger x eksponentielt.

```
----- OPGAVE 5 -----  
N: 20  
x 400  
N: 40  
x 1440  
N: 80  
x 4480  
-----
```

```
> N = 10, så  
k: 16  
j: 11  
i: 4  
x 120
```

$k = 0 \rightarrow k = 1 \rightarrow k = 2 \rightarrow k = 4 \rightarrow k = 8 \rightarrow k = 16$ ($k > N = 10$, så løkken fejler her)

j incrementer med 1 hver gang og fejler ved 11

i = 0 -> i = 1 -> i = 2 -> i = 3 -> i = 4 (i > 3, så løkken fejler her)

De multipliceres til $O(N^{1/2}) * O(N) * O(\log N) \Rightarrow O(N^{3/2} * \log N)$

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 6

```
int PlExercises::sumDivisibleBy3(int N) {
    if (N < 3){
        return 0; // base case
    }

    if (N % 3 == 0){
        return N + sumDivisibleBy3(N - 3); //now we can just remove 3 each time
    }

    return sumDivisibleBy3(N - 1); //progress
}
```

----- OPGAVE 6 -----

```
N: 12
sum3(N) = 30
N: 14
sum3(N) = 30
```

Opgave 7

Minimums-værdierne for X og Y er bestemt til at være 3, og maks- Z er sat til 100 000, så maks værdier for X og Y kan bestemmes ved:

- største værdi af X ved mindste Y, når $X^3 < 100\,000 \Rightarrow 46$
- største værdi af Y ved mindste X, når $3^Y < 100\,000 \Rightarrow 10$

```
std::vector<int> isExponent(int Z) {
    if(Z > 100000 || Z <= 0){
        return {-1, -1}; //error
    }

    /*initialisere værdier*/
    int minX = 3; //for X > 2
    int minY = 3; //fordi Y > 2

    int maxX = 46; //for X³ < Z = 100 000
    int maxY = 10; //for (X = 3); 3¹⁰ < Z = 100 000

    for(int y = minY; y <= maxY; y++){
        for(int x = maxX; x>=minX; x--){ // decrement for at finde
            største x (start i maxx)
            if(Z == pow(x, y)){
                return {x, y};
            } else if(pow(x, y) > Z){ //prøv mindre x værdi
                continue;
            }
        }
    }

    return {-1, -1}; //ingen løsning
}
```

----- OPGAVE 7 -----

```
Z: 6561
Z = 9⁴
```

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 8

Indeks	Værdi	Indeks	Værdi	Indeks	Værdi	Indeks	Værdi
0		0		0		0	H
1		1		1		1	
2	V	2	V	2	V	2	V
3	R	3	R	3	R	3	R
4		4		4		4	
5		5		5		5	
6	P	6	P	6	P	6	P
7		7	Q	7	Q	7	Q
8	E	8	E	8	E	8	E
9		9		9	c	9	c
10	F	10	F	10	F	10	F

(0) Original tabel

(1) Q hasher til indeks 7

(2)

(3)

(2) C hasher til indeks 8 som er optaget,

Bruger quadratic probing:

1. Iteration: $(\text{hash}(x) + 1 * 1) \% \text{TableSize} \Rightarrow (8 + 1) \% 11 = 9$ (indeks ledigt)

(3) H hasher til indeks 2 som er optaget

1. Iteration: $(2 + 1 * 1) \% \text{TableSize} \Rightarrow 3$ (indeks optaget)
2. $(2 + 2 * 2) \% 11 \Rightarrow 6$ (indeks optaget)
3. $(2 + 3 * 3) \% 11 \Rightarrow 0$ (indeks ledigt)

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 9

```
long myMethod3(int n){
    if(n <= 1){
        return 1;
    }else{
        std::cout<<"n: " << n << "n-1: " << myMethod3(n-1)
        <<"n-2: " << myMethod3(n-2) << std::endl;
        return myMethod3(n-1) + myMethod3(n-2);
    }
}
```

Dette svarer til/genkendes som en rekursiv fibonacci metode.

(kan bla ses ved udskrifter af de rekursive kald der har en træ-struktur)

Tidskompleksiteten her er eksponentiel altså: $O(2^N)$

```
----- OPGAVE 9 -----
N: 5
n: 5n-1: n: 4n-1: n: 3n-1: n: 2n-1: 1n-2: 1
2n-2: 1
n: 2n-1: 1n-2: 1
3n-2: n: 2n-1: 1n-2: 1
2
n: 3n-1: n: 2n-1: 1n-2: 1
2n-2: 1
n: 2n-1: 1n-2: 1
n: 2n-1: 1n-2: 1
5n-2: n: 3n-1: n: 2n-1: 1n-2: 1
2n-2: 1
n: 2n-1: 1n-2: 1
3
n: 4n-1: n: 3n-1: n: 2n-1: 1n-2: 1
2n-2: 1
n: 2n-1: 1n-2: 1
3n-2: n: 2n-1: 1n-2: 1
2
n: 3n-1: n: 2n-1: 1n-2: 1
2n-2: 1
n: 2n-1: 1n-2: 1
n: 2n-1: 1n-2: 1
n: 3n-1: n: 2n-1: 1n-2: 1
2n-2: 1
n: 2n-1: 1n-2: 1
long ret = P1.my
std::cout << "re
returned: 8
```

Opgave 10

```
int logTwo(int N){
    if (N == 1){ // 2^0 = 1
        return 0;
    }
    if(N <= 0 || N%2 != 0){
        return -1; //error
    }

    int b = logTwo(N/2); //divide and conquer recursive call
    int y = b+1;

    return y;
}
```

```
----- OPGAVE 10 -----
N: 32
Log2(N) = 5
N: 4096
Log2(N) = 12
```


28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Opgave 11

(Første implementering af algoritmen)

```
int majorityVotes(std::vector<int> votes){
    if(votes.empty()){
        return -1; //no votes or candidates
    }
    int nVotes = votes.size(); //length of array
    /*initialize each candidate to 0 votes*/
    int nCandidates = *std::max_element(votes.begin(), votes.end());
    std::vector<int> voteCounts(nCandidates + 1, 0);

    /*add up votes*/
    for (int vote : votes){
        if(vote >= 0 && vote <= nCandidates){ //can only vote for "valid" candidates
            voteCounts[vote]++;
        }
    }
    /*find best candidate*/
    auto it = std::max_element(voteCounts.begin(), voteCounts.end());
    int bestCandidate = (int)std::distance(voteCounts.begin(), it);
    int maxVotes = *it;

    std::cout<<"bestCandidate: "<<bestCandidate<<std::endl;
    std::cout<<"maxVotes: "<<maxVotes<<std::endl;

    /*more than 50% of votes?*/
    return (maxVotes > nVotes / 2) ? bestCandidate : -1;
}
```

```
----- OPGAVE 11 -----
original votes:
7, 4, 3, 5, 3, 1, 6, 4, 5, 1, 7, 5, 98,
bestCandidate: 5
maxVotes: 3
returned = -1
-----
original votes:
7, 7, 7, 7, 7, 7, 6, 4, 5, 1, 7, 5, 98,
bestCandidate: 7
maxVotes: 7
returned = 7
```

Denne bruger `std::max_element`, som har **$O(N)$ tidskompleksitet** når den bruges på array `votes` med størrelse N .

Den første for-løkke gennemløber også array'et [størrelse N] og har også **$O(N)$ tidskompleksitet**

`std::max_element` bruges anden gang på "`voteCounts`" som har størrelsen $K = N + 1$ (der er en tæller per kandidat), og dette metode kald har derfor **$O(K)$ tidskompleksitet**

Derfor er tidskompleksiteten: $O(N + K)$

Implementering af boyer-moore-majority-vote algoritmen som et andet løsnings-forslag.

Bla fordi jeg var usikker på om brug af indbyggede sorterings-metoder er tilladt.

(med udgangspunkt i gennemgangen herfra:

<https://www.geeksforgeeks.org/theory-of-computation/boyer-moore-majority-voting-algorithm/>)

(Boyer-moore algoritmen er vidstnok også referet til i bogen: side 75)

```
int majorityVotesBoyerMoore(std::vector<int> votes){
    if(votes.empty()){
        return -1; //no votes or candidates
    }
    /*finder bedste kandidat, count */
    int candidate = -1; //ingen kandidat
    int balance = 0;
    //int candidates = votes.size();
    for(int vote : votes){
        if(balance == 0){
            candidate = vote; //gem værdi
            balance = 1; //start
        }else if(vote == candidate){
            balance++; //tæl op
        }else {
            balance--; //tæl ned, kandidater udligner hinanden
        }
    }
    /*verificer om kandidat er majority*/
    int voteCount = 0;
    for(int vote : votes){
        if(vote == candidate){
            voteCount++; //tæller hvor mange votes bedste kandidat har
        }
    }
    return (voteCount > (int)votes.size()/2) ? candidate : -1;
}
```

28/10/2025

Lina Langemark

5. semester civilingeniør i robotteknologi bachelor

Ideen er at finde en "flertalskandidat" ved at holde styr på en "balance" [tæller] mellem antallet af gange den bedste kandidat er set, og antallet af gange andre kandidater er set. (de 2 tal "trækkes fra hinanden").

Første for-løkke: En flertals-kandidat findes, når balancen bliver nul skiftes kandidat, hvis der findes et flertal, vil balancen være mere end nul. For-løkken iterere henover "votes" med størrelsen N

—> **$O(N)$ tidskompleksitet**

Anden for-løkke: Efterfølgende tjekkes/verificeres det om kandidatens "votes" er et flertal, ved at tælle hvor mange votes kandidaten har. Samme tabel-størrelse

—> **$O(N)$ tidskompleksitet**

Derfor samlet tidskompleksitet: $O(N)$

```
----- OPGAVE 11: Boyer Moore -----  
original votes:  
7, 4, 3, 5, 3, 1, 6, 4, 5, 1, 7, 5, 98,  
returned = -1  
-----  
original votes:  
7, 7, 7, 7, 7, 7, 6, 4, 5, 1, 7, 5, 98,  
returned = 7
```